

# RIM DEFECT DETECTION

Complete Unified Reference

GPU Notebook v4.0 · TPU Notebook · Mathematics Reference

All three guides merged, cross-referenced, and matched section-by-section

Updated May 2026 · 28 Notebook Sections · 27 Math Sections · 12 TPU Change  
Sections

## How to Read This Document

This PDF merges three source documents into a single unified reference. Every notebook section now contains:

| Element                | What it contains                                                                         |
|------------------------|------------------------------------------------------------------------------------------|
| Notebook Guide text    | The plain-English explanation from the Complete Notebook Guide (GPU v4.0 baseline)       |
| Mathematics            | The exact formulae from the Mathematics Reference, matched to the section they apply to  |
| TPU changes            | Every change between the GPU v4.0 and TPU notebooks, matched to the section they affect  |
| Cross-reference badges | Green GPU §N · Purple TPU §N · Teal MATH §N badges link related content across documents |
| Comparison tables      | Side-by-side GPU vs TPU tables wherever a parameter or pattern changed                   |
| Bug fix boxes          | Green FIX boxes calling out the 24 known bugs fixed across both notebooks                |

GPU §N

→ GPU Notebook v4.0 section

TPU §N

→ TPU Notebook change section

MATH §N

→ Mathematics Reference section

# Table of Contents

## PART 1 — NOTEBOOK GUIDE — GPU v4.0 & TPU

- 1 What Are We Building? — Two-Stage Cascade
- 2 The Dataset — Class Imbalance & Splits
- 3 Environment Setup & Imports
- 4 Configuration (CLI Arguments)
- 5 Stage 1: EfficientNet-B0 — The Fast Filter
- 6 Custom Augmentation Classes
- 7 Image Transforms — Light vs Heavy Pipelines
- 8 Dataset Class (RimDataset)
- 9 Data Loading — Samplers & Loaders
- 10 Attention Modules — CBAM & AttentionPool2d
- 11 EnsembleModelV6 — Three-Backbone Fusion
- 12 Loss Functions — Focal, ASL, Combined
- 13 EMA — Exponential Moving Average
- 14 MixUp / CutMix
- 15 TTA — Test-Time Augmentation
- 16 Validation & Threshold Sweep
- 17 Batch Normalisation Recalibration
- 18 Platt Scaling — Probability Calibration
- 19 Three-Phase Progressive Unfreezing
- 20 AMP — Automatic Mixed Precision
- 21 Training Loop — Full Walkthrough
- 22 Cascade Inference Pipeline
- 23 Pseudo-Labeling
- 24 Evaluation & Metrics Report
- 25 Known Bugs Fixed — All 24 Issues
- 26 OHEM — Online Hard Example Mining
- 27 Grad-CAM Dynamic Cropping (GPU only)
- 28 Glossary

## PART 2 — TPU CHANGES — GPU v4.0 → TPU Notebook

- T1 Overview — 10 Core Changes at a Glance
- T2 Environment Setup — torch\_xla Replaces CUDA
- T3 Configuration — Batch, Accumulation, Precision
- T4 Stage 1 — Bigger Batches, Native bfloat16
- T5 Data Loading — DistributedSampler & MpDeviceLoader
- T6 Augmentations — Two New Transforms Added
- T7 Rim Cropping — HoughCircles Replaces Fixed Coords
- T8 Training Loop — xm.\* API
- T9 Validation & BN Recalibration
- T10 Inference Pipeline — Grad-CAM Removed
- T11 Multi-Process Entry Point — xmp.spawn
- T12 Complete Hyperparameter Reference

## PART 3 — MATHEMATICS REFERENCE — All 27 Sections

- M1 Image Normalisation
- M2 Data Augmentation Mathematics
- M3 Oversampling & Class Weighting
- M4 Attention Mechanisms — CBAM

- M5 Attention Pooling (AttentionPool2d)
- M6 Cross-Backbone Multi-Head Attention
- M7 Activation Functions
- M8 Loss Functions — Full Derivations
- M9 Online Hard Example Mining (OHEM)
- M10 Exponential Moving Average (EMA)
- M11 MixUp & CutMix
- M12 Test-Time Augmentation — D4 Dihedral Group
- M13 Optimiser — AdamW
- M14 Learning Rate Schedule — Cosine Annealing
- M15 Gradient Clipping
- M16 Gradient Accumulation & Effective Batch Size
- M17 Mixed Precision & Loss Scaling
- M18 Batch Normalisation & Recalibration
- M19 Stochastic Weight Averaging (SWA)
- M20 Platt Scaling
- M21 Evaluation Metrics
- M22 Threshold Sweep — Vectorised F1
- M23 Cascade Inference Decision Rule
- M24 Rim Cropping — HoughCircles
- M25 Grad-CAM (GPU Only)
- M26 Pseudo-Labelling Confidence Filter
- M27 Vignette, Ring & Specular Highlight Geometry

## PART 1

# Notebook Guide

GPU v4.0 baseline — with TPU changes and maths matched inline

# 1 What Are We Building?

MATH §M23

We are building a **computer vision system** that automatically inspects car wheel (rim) photos and decides: *Is this rim defective or not?* Defects include scratches, cracks, dents, and surface anomalies.

## The Two-Stage Cascade Approach

Think of this like airport security:

| Stage                  | Model                                    | Role                                                                                                     |
|------------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Stage 1 (Fast Scanner) | EfficientNet-B0 — lightweight, 5M params | Looks at every full image. If VERY confident of a defect → flags immediately. Input: 384×384 full image. |
| Stage 2 (Full Scan)    | EnsembleModelV6 — 50M+ params            | Only runs on images Stage 1 was unsure about. Input: 256×256 cropped rim.                                |

### Cascade Decision Rule (MATH §M23):

```
p_B0 = sigma( B0(full_image) )
p_ensemble = mean_k( sigma( Ensemble_k(crop) ) ) # over k folds

p_final = p_B0 if p_B0 >= tau_1 (0.70)
          = p_ensemble otherwise

y_hat = 1 if p_final >= tau_blend, else 0
```

## 2 The Dataset

### MATH §M3

Images are 1280×1024 RGB photographs of car rims from a fixed camera. The rim covers ~40% of the image; the rest is background.

### Class Imbalance

| Class                | Fraction | Approx count   |
|----------------------|----------|----------------|
| Good (negative)      | 82.4%    | ~29,000 images |
| Defective (positive) | 17.6%    | ~6,200 images  |

### Two fixes applied (MATH §M3):

1. **Oversampling** — minority class duplicated until 50/50 balance:

```
n_extra = floor(n0 * r) - n1 # r = 1.0 (target ratio)
extra[i] ~ UniformChoice(minority_samples) # with replacement
Result: both classes equal; pos_weight = 1.0
```

2. **WeightedRandomSampler** (alternative to oversampling):

```
w0 = 1/n0 w1 = 1/n1
P(sample i drawn) = w[label_i] / sum_j w[label_j]
```

■ **NOTE** 5-fold cross-validation: data split into 5 folds, 5 separate models trained (each fold as validation once). Final prediction = average of 5 models.

### 3 Environment Setup & Imports

TPU §T2

The setup cell installs libraries, configures memory, and imports everything needed.

#### Critical: CUDA memory config (GPU only)

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:64"
```

Prevents fragmented allocations on the 15 GB Kaggle T4 GPU. Not needed on TPU — removed in TPU notebook.

#### GPU vs TPU Setup

| Feature       | GPU Notebook v4.0                                             | TPU Notebook                                              |
|---------------|---------------------------------------------------------------|-----------------------------------------------------------|
| Device setup  | <code>torch.device("cuda")</code>                             | <code>xm.xla_device()</code> — one per spawned process    |
| Memory config | <code>PYTORCH_CUDA_ALLOC_CONF = "max_split_size_mb:64"</code> | Removed — XLA manages its own memory                      |
| New imports   | —                                                             | <code>torch_xla, xm, xr, xmp, pl (parallel_loader)</code> |
| New helpers   | —                                                             | <code>get_device(), get_num_tpu_chips()</code>            |

#### ImageNet Normalisation Constants (MATH §M1):

```
IMAGENET_MEAN = [0.485, 0.456, 0.406] # R, G, B
IMAGENET_STD = [0.229, 0.224, 0.225]

x_norm[c] = (x[c] / 255 - mean[c]) / std[c]
```



4 Configuration (CLI Arguments)

TPU §T3

All hyperparameters are set via argparse. On Kaggle (no command line), every argument uses its default value.

Key Arguments — Training Schedule

| Argument        | GPU Default | TPU Default | Purpose                     |
|-----------------|-------------|-------------|-----------------------------|
| --phase1-epochs | 5           | 5           | Train head only             |
| --phase2-epochs | 8           | 8           | Head + last backbone blocks |
| --phase3-epochs | 27          | 27          | Full model unfreeze         |
| --early-stop    | 12          | 12          | Patience epochs             |
| --swa-start     | 30          | 30          | Epoch to start SWA          |

Key Arguments — Hyperparameters with GPU/TPU Differences

| Feature           | GPU Notebook v4.0 | TPU Notebook                                            |
|-------------------|-------------------|---------------------------------------------------------|
| --batch           | 2                 | 192 (8 chips × 24)                                      |
| --accum-steps     | 32                | 1 (real batch 192 needs none)                           |
| --label-smooth    | 0.02              | 0.01 (large batch already regularises)                  |
| --mixup-alpha     | 0.10              | 0.20 (more aggressive)                                  |
| --focal-gamma     | 2.0               | 2.5 (harder focus on difficult examples)                |
| --ema-decay       | 0.9998            | 0.9999 (slower; each step more reliable at large batch) |
| --bf16            | False             | True (TPU native; always on)                            |
| --channels-last   | True              | False (not applicable on TPU)                           |
| --compile         | False             | False (torch.compile unsupported on XLA)                |
| --grad-checkpoint | True              | False (16 GB/chip sufficient at batch 192)              |
| --workers         | 2                 | 4                                                       |

■ **NOTE** Effective batch: GPU = 2 × 32 = 64 (accumulated). TPU = 192 × 1 = 192 (real). See MATH §M16.

5

Stage 1: EfficientNet-B0 — The Fast Filter

TPU §T4

MATH §M23

EfficientNet-B0 (5M parameters) is a separate, lighter model that pre-filters images before the expensive ensemble runs. It trains on **full uncropped images** (1280×1024 → 384×384), unlike the ensemble which sees only the cropped rim.

Stage 1 Training Setup

| Property     | Value                                                          |
|--------------|----------------------------------------------------------------|
| Architecture | EfficientNet-B0 (pretrained ImageNet)                          |
| Loss         | BCEWithLogitsLoss, pos_weight=1.0 (dataset oversampled to 1:1) |
| Optimizer    | AdamW, lr=1e-4, weight_decay=1e-4                              |
| Schedule     | Cosine annealing with linear warmup                            |
| Input size   | 384×384 full image (not rim crop)                              |
| Threshold    | STAGE1_THRESHOLD = 0.70                                        |

GPU vs TPU Differences in Stage 1

| Feature          | GPU Notebook v4.0           | TPU Notebook                                |
|------------------|-----------------------------|---------------------------------------------|
| STAGE1_BATCH     | 8                           | 128                                         |
| autocast backend | torch.autocast('cuda', ...) | torch.autocast('xla', dtype=torch.bfloat16) |
| Optimizer step   | optimizer.step()            | xm.optimizer_step(optimizer)                |
| Checkpoint save  | torch.save()                | xm.save() (handles XLA tensor transfer)     |
| pin_memory       | True                        | False                                       |

✓ **FIX BUG #8 FIXED:** Original code oversampled to 1:1 AND used pos\_weight=9.0 — the model guessed defective on everything. Fix: set pos\_weight=1.0 when oversampling is enabled.

6

Custom Augmentation Classes

TPU §T6

MATH §M2

Five custom augmentations simulate real-world rim photography artefacts. All run on **CPU** before tensors reach the device — the correct pattern for both GPU and TPU (keeps MXUs/CUDA cores on matrix ops).

| Class             | Mechanism                        | Simulates                                  | Math           |
|-------------------|----------------------------------|--------------------------------------------|----------------|
| RadialBlur        | Diagonal motion blur kernel      | Simulates rim spinning during imaging.     | MATH §M2e      |
| SpecularHighlight | 2D Gaussian ellipse added        | Metal surface glare from factory lighting. | MATH §M2a      |
| ScratchSimulation | Trigonometric line placement     | Random thin lines simulating scratches.    | MATH §M2e      |
| SectorMask        | Pie-slice filled with mean color | Occlusion / partial rim capture.           | MATH §M2b      |
| RimSpecificAug    | Vignette / ring / fixture bar    | Lighting rigs, camera artefacts.           | MATH §M2c, M2d |

New in TPU Notebook (MATH §M2):

- A.GridDistortion(p=0.25) — Local grid-based warping. Simulates lens distortion.
- A.RandomShadow(p=0.20) — Random dark polygon. Simulates fixture shadows.

■ **NOTE** These were added because Phase 3 on TPU (batch 192, all layers unfrozen) sees far more gradient signal per epoch. More aggressive augmentation prevents overfitting.

Key Formulae

Specular Highlight (MATH §M2a):

```
mask(x,y) = exp( -[ ((x-cx)/rx)^2 + ((y-cy)/ry)^2 ] )
pixel_out[c] = clip( pixel_in[c] + mask * intensity * 255 , 0, 255 )
intensity ~ Uniform(0.2, 0.7)
```

SectorMask (MATH §M2b):

```
r_bound = ceil( sqrt((W/2)^2 + (H/2)^2) ) + 1
start_angle ~ Uniform(0, 360) sweep_angle ~ Uniform(10, 60)
```

7 Image Transforms

MATH §M1

Two transform pipelines: training (with randomness) and validation (deterministic).

\_make\_train\_transform — Light vs Heavy

| Transform                 | Light Mode   | Heavy Mode   |
|---------------------------|--------------|--------------|
| Used in phases            | Phases 1 & 2 | Phase 3 only |
| Resize+flips              | ✓            | ✓            |
| BrightnessContrast        | ✓            | ✓            |
| ShiftScaleRotate          | ✗            | ✓            |
| ElasticTransform          | ✗            | ✓            |
| CLAHE / HueSat / Gamma    | ✗            | ✓            |
| Noise (Gauss/ISO/Mult)    | ✗            | ✓            |
| CoarseDropout             | ✗            | ✓            |
| All 5 custom augs         | ✗            | ✓            |
| GridDistortion (TPU only) | ✗            | ✓ (TPU)      |
| RandomShadow (TPU only)   | ✗            | ✓ (TPU)      |

make\_val\_transform — No randomness:

```
A.Resize(img_size, img_size)
A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)
A.ToTensorV2()
```

■ **NOTE** Validation transform is identical for both GPU and TPU notebooks. NO augmentation — ensures deterministic metrics every run.

## 8 Dataset Class — RimDataset

RimDataset is a standard PyTorch Dataset. The 'waiter' for the training kitchen: given an index, it fetches the image and its label.

### Constructor parameters

| Parameter | Purpose                                                         |
|-----------|-----------------------------------------------------------------|
| samples   | List of (path, label) tuples                                    |
| transform | Augmentation pipeline (augmentations or torchvision)            |
| img_size  | Target size (default 300, overridden by args.img_size)          |
| cache     | If True, preloads ALL images into RAM at startup (fast for val) |

### getitem flow:

```
1. path, label = self.samples[idx]
2. img = cache[idx] OR Image.open(path).convert('RGB') → np.array
3. if augmentations: tensor = transform(image=img_arr)['image']
   else: tensor = transform(Image.fromarray(img_arr))
4. return tensor, torch.tensor(label, dtype=torch.float32)
```

■ **WARNING** Warning: Cache only large val sets if RAM permits. >5000 images triggers a RAM warning.

9 Data Loading — Samplers & Loaders

TPU §T5      MATH §M3

Four helper functions manage data: load\_samples → split\_samples → oversample\_minority → make\_loaders.

GPU vs TPU DataLoader Differences

| Feature            | GPU Notebook v4.0            | TPU Notebook                                        |
|--------------------|------------------------------|-----------------------------------------------------|
| pin_memory         | True                         | False — XLA manages its own memory                  |
| Shuffle            | shuffle=True in DataLoader   | DistributedSampler(shuffle=True)                    |
| Sampler            | WeightedRandomSampler        | DistributedSampler — divides dataset across 8 chips |
| Device loader      | DataLoader iterated directly | pl.MpDeviceLoader wraps every loader                |
| persistent_workers | True                         | False — XLA handles prefetch                        |
| prefetch_factor    | 2                            | Removed (XLA manages prefetching)                   |

DistributedSampler ensures chip k only sees 1/8 of the data each step:

```
train_sampler = DistributedSampler(
    train_dataset,
    num_replicas=world_size, # 8
    rank=rank, # 0-7
    shuffle=True,
)
```

MpDeviceLoader prefetches the next batch to the XLA device asynchronously:

```
xla_loader = pl.MpDeviceLoader(loader, device)
for imgs, lbls in xla_loader:
    ... # imgs already on XLA device
```

## 10 Attention Modules — CBAM & AttentionPool2d

MATH §M4,M5

Attention lets the model focus on important parts of images — like how your eyes go to a scratch, not the clean metal around it.

### CBAM — Convolutional Block Attention Module (MATH §M4)

#### Step 1 — Channel Attention: Which feature channels matter?

```
F_avg = AvgPool(X) → (B, C)
F_max = MaxPool(X) → (B, C)
MLP(v) = W2 * ReLU( W1 * v ) W1:(C, C/16) W2:(C/16, C)
M_c = sigma( MLP(F_avg) + MLP(F_max) ) → (B, C)
X' = X * M_c.unsqueeze(-1).unsqueeze(-1)
```

#### Step 2 — Spatial Attention: Which pixels matter?

```
sp_in = cat( mean(X', dim=C), max(X', dim=C) ) → (B, 2, H, W)
M_s = sigma( Conv7x7(sp_in) ) → (B, 1, H, W)
X'' = X' * M_s
```

### AttentionPool2d — Replaces Global Average Pooling (MATH §M5)

Standard GAP averages ALL pixels equally — a crack covering 1% is diluted by 99% clean metal. AttentionPool learns WHERE to look:

```
scores = Conv1x1(X) → (B, 1, H, W)
w = softmax( scores.flatten(dim=2) ) → (B, 1, H*W)
AttPool(X) = sum_k w[:, 0, k] * X_flat[:, :, k] → (B, C)
```





## 12 Loss Functions

MATH §M8,M9

### Focal Loss (MATH §M8c):

```
pt = p * y + (1-p) * (1-y) p = sigma(logit)
FocalLoss = (1 - pt)^gamma * BCE(logit, y)
gamma = 2.0 (GPU) / 2.5 (TPU)
```

When  $pt \rightarrow 1$  (easy correct):  $(1-pt)^2 \rightarrow 0$  (loss suppressed)  
 When  $pt \rightarrow 0$  (hard wrong):  $(1-pt)^2 \rightarrow 1$  (loss preserved)

### Asymmetric Loss (MATH §M8d):

```
gamma_pos = 1 gamma_neg = 4 (harder focus on easy negatives)
p_clip = clip(p + 0.05, max=1-eps)
L_pos = -y * log(clip(p, min=eps))
L_neg = -(1-y) * log(1 - p_clip)
w_asl = (1-pt)^gamma_pos if y>=0.5 else (1-pt)^gamma_neg
ASL = -(L_pos + L_neg) * w_asl
```

### CombinedLoss — Full Training Objective (MATH §M8e):

```
L_bin = 0.60 * Focal + 0.40 * ASL
L_sev = MSE( sigma(z_sev), y ) = (sigma(z_sev) - y)^2
L_total = L_bin + 0.25 * L_sev
```

### Label Smoothing (MATH §M8b):

```
y_smooth = y*(1-eps) + 0.5*eps
eps=0.02 (GPU) / 0.01 (TPU)
Example: y=1 → 0.99 (GPU), 0.995 (TPU)
```

## 13 Exponential Moving Average (EMA)

MATH §M10

EMA maintains a shadow copy of the model whose weights are a running average of all past states. The EMA model almost always outperforms the raw model.

```
d_t = min(decay, (1+t)/(10+t)) # warmup ramp
shadow[t+1] = d_t * shadow[t] + (1-d_t) * theta[t]

decay = 0.9998 (GPU) / 0.9999 (TPU)

Effective averaging window = 1/(1-decay)
GPU: 1/(1-0.9998) = 5,000 steps
TPU: 1/(1-0.9999) = 10,000 steps
```

■ **NOTE** TPU uses slower EMA (0.9999) because each gradient step is computed on more data (batch 192 vs 64 effective), so individual steps are more reliable and the EMA should be more conservative.

## 14 MixUp / CutMix

MATH §M11

Both techniques create hybrid training images to prevent memorisation. Used in Phases 2 and 3 only.

```
lambda ~ Beta(alpha, alpha) alpha=0.10 (GPU) / 0.20 (TPU)

MixUp (50% probability):
x_mix = lambda*x + (1-lambda)*x[idx]
y_mix = lambda*y + (1-lambda)*y[idx]

CutMix (50% probability):
cut_r = sqrt(1-lambda) cut_h = H*cut_r cut_w = W*cut_r
x_mix[:, :, y1:y2, x1:x2] = x[idx, :, :, y1:y2, x1:x2]
lambda_actual = 1 - (x2-x1)*(y2-y1)/(H*W) # true area ratio
y_mix = lambda_actual*y + (1-lambda_actual)*y[idx]
```

■ **NOTE** TPU uses alpha=0.20 (more aggressive). Larger effective batch (192) means more gradient signal, so stronger augmentation is needed to prevent overfitting.

## 15 Test-Time Augmentation (TTA)

MATH §M12

At inference, the image is run through the model 8 times (all D4 symmetries) and predictions are averaged. Identical in both GPU and TPU notebooks.

```
D4 elements: identity, rot90, rot180, rot270,  
flipH, flipH+rot90, flipV, flipV+rot90  
  
p_TTA = (1/8) * sum_{g in D4} sigma( model( g(x) ) )  
y_hat = 1 if p_TTA >= threshold, else 0  
  
Averaging 8 views reduces prediction variance.
```

■ **NOTE** TTA is optional during training validation (--tta-freq controls frequency). Always used for final inference. 8× slower than single inference.

## 16 Validation & Threshold Sweep

MATH §M21,M22

Instead of the default 0.5 threshold, the optimal threshold is found by vectorised F1 sweep across all thresholds 0.05–0.95 simultaneously on the GPU/TPU (MATH §M22):

```
probs shape: (N,)
thrs shape: (91,) linspace(0.05, 0.95, step=0.01)
preds = (probs.unsqueeze(0) >= thrs.unsqueeze(1)) # (91, N)

TP = (preds * labels).sum(1)
FP = (preds * (1-labels)).sum(1)
FN = ((1-preds) * labels).sum(1)
P = TP / clip(TP+FP, min=1)
R = TP / clip(TP+FN, min=1)
F1 = 2*P*R / clip(P+R, min=1e-8)
best_threshold = thrs[ argmax(F1) ]
```

### Evaluation Metrics (MATH §M21):

```
Accuracy = (TP+TN) / (TP+TN+FP+FN)
Precision = TP / (TP+FP) 'of predicted defects, how many are real?'
Recall = TP / (TP+FN) 'of all real defects, how many caught?'
F1 = 2*P*R / (P+R) 'harmonic mean – main metric'
AUC-ROC = P(score(defective) > score(clean)) [ranking quality]
```

## 17 Batch Normalisation Recalibration

TPU §T9

MATH §M18

EMA/SWA models have averaged weights but stale BatchNorm running statistics. Recalibration resets and re-estimates them using 25 training batches.

```
# BN forward pass (training mode) – MATH §M18
mu_B = (1/B) * sum_i x_i
sigma_B = sqrt( (1/B)*sum_i(x_i-mu_B)^2 + eps )
x_hat_i = (x_i - mu_B) / sigma_B
y_i = gamma * x_hat_i + beta

# Running stats update:
mu_run <- (1-momentum)*mu_run + momentum*mu_B
sigma_run <- (1-momentum)*sigma_run + momentum*sigma_B

# Recalibration: reset + re-estimate with momentum=None
# (cumulative moving average: momentum = 1/num_batches_tracked)
```

■ **NOTE** TPU change: BN recalibration loader wrapped in `pl.MpDeviceLoader`. `xm.mark_step()` called after each of the 25 recalibration batches.

## 18 Platt Scaling — Probability Calibration

**MATH §M20**

Neural networks are often overconfident. Platt Scaling fits a logistic regression on validation outputs to make probabilities match reality.

```
calibrated_prob = sigma( A * raw_prob + B )

Fitted by minimising:
sum_i BCE( sigma(A*p_i + B), y_i ) over validation set

Using sklearn LogisticRegression:
cal.fit( probs.reshape(-1,1), labels )
A = cal.coef_[0,0] B = cal.intercept_[0]

Saved to: calibrator_fold_{n}.pkl
```

■ **NOTE** After calibration: if model says 90%, ~90% of those images truly are defective. Without calibration the model may say 95%+ but only be 80% correct.

## 19 Three-Phase Progressive Unfreezing

MATH §M13,M14

Progressive unfreezing prevents destroying pretrained backbone knowledge before the head learns to use it.

| Phase   | Epochs    | Trainable Layers                                       | Augmentation | Learning Rate                             |
|---------|-----------|--------------------------------------------------------|--------------|-------------------------------------------|
| Phase 1 | 5 epochs  | Head only (bin_head, sev_head, classifier, cross_attn) | Light        | base_lr                                   |
| Phase 2 | 8 epochs  | Head + last 2 blocks of each backbone                  | Light        | Head: base_lr Backbone: 0.1×base_lr       |
| Phase 3 | 27 epochs | Everything unfrozen                                    | Heavy        | Head: base_lr Backbone: 0.02-0.04×base_lr |

### Cosine Annealing LR Schedule (MATH §M14):

```
lr_t = lr_min + 0.5*(lr_max-lr_min) * (1 + cos(pi*t/T_max))
lr_max = 5e-5 lr_min = lr_max * 0.01 = 5e-7
T_max = epochs in current phase (5, 8, or 27)
```

### AdamW Optimiser (MATH §M13):

```
m_t = beta1*m_{t-1} + (1-beta1)*g_t beta1=0.9
v_t = beta2*v_{t-1} + (1-beta2)*g_t^2 beta2=0.999
theta_{t+1} = theta_t - lr*m_hat/sqrt(v_hat+eps) - lr*wd*theta_t
wd = 1e-4 eps = 1e-8
```



20 AMP — Automatic Mixed Precision

TPU §T3      MATH §M17

Mixed precision uses 16-bit numbers for most calculations and 32-bit only where needed — 2× faster, half the memory.

| Feature                     | GPU Notebook v4.0                 | TPU Notebook                                |
|-----------------------------|-----------------------------------|---------------------------------------------|
| GPU Ampere+ (A100, L4)      | bfloat16 — stable, no GradScaler  | bfloat16 native — always on                 |
| GPU Volta/Turing (T4, V100) | float16 + GradScaler              | Not applicable                              |
| CPU                         | No AMP                            | Not applicable                              |
| autocast call               | torch.autocast('cuda', dtype=...) | torch.autocast('xla', dtype=torch.bfloat16) |
| GradScaler                  | Required for float16              | Removed — bfloat16 needs no scaler          |

Why bfloat16 needs no scaler (MATH §M17):

```
float16: 5 exponent bits range ≈ 6e-5 to 65504
→ small gradients underflow to 0
→ GradScaler multiplies loss by 2^16 before backward

bfloat16: 8 exponent bits same range as float32 (2^-126 to 2^127)
→ no underflow risk → no GradScaler needed
```

## 21 Training Loop — Full Walkthrough

TPU §T8

MATH §M15,M16

The full training loop runs for all phases, all epochs, all batches. Core structure:

```
for fold in 5_folds:
    for phase, n_epochs in [(1,5),(2,8),(3,27)]:
        setup_optimizer_and_scheduler(phase)
        for epoch in range(n_epochs):
            model.train()
            for step, (imgs, lbls) in enumerate(loader):
                imgs, lbls = MixUp_CutMix(imgs, lbls) # phases 2+
                with autocast:
                    bin_logits, sev_logits = model(imgs)
                    loss = criterion(...) / accum_steps
                    [scaler.scale(loss) if fp16 else loss].backward()
                    if (step+1) % accum_steps == 0:
                        clip_grad_norm_(model.params, max_norm=0.5)
                        [scaler.step(opt) | xm.optimizer_step(opt) | opt.step()]
                        ema.update(model)
                        [xm.mark_step() # TPU only]
                        scheduler.step()
            # validation, threshold sweep, early stopping
```

### Gradient Clipping (MATH §M15):

```
g_norm = sqrt( sum_i ||grad_i||^2 )
if g_norm > 0.5: grad_i <- grad_i * (0.5 / g_norm)
```

### Gradient Accumulation (MATH §M16):

```
loss_step = L_total(batch) / accum_steps
effective_batch = batch * accum_steps
GPU: 2 * 32 = 64 TPU: 192 * 1 = 192
```

### xm.mark\_step() — Critical TPU Concept (TPU §T8):

XLA compiles operations lazily. None of them execute until `mark_step()` is called. `xm.optimizer_step()` calls it internally. Must also be called explicitly after EMA updates and after each validation batch.

✓ **FIX BUG #15 FIXED:** Optimizer is only restored from checkpoint when the checkpoint phase matches the current training phase. Restoring a Phase 2 optimizer during Phase 1 caused a crash.

22

Cascade Inference Pipeline

TPU §T10

MATH  
§M23,M24,M25

At inference, Stage 1 acts as a gate. Only uncertain images reach the expensive ensemble.

| Feature           | GPU Notebook v4.0                                               | TPU Notebook                                                 |
|-------------------|-----------------------------------------------------------------|--------------------------------------------------------------|
| Rim crop method   | Fixed relative coordinates + optional Grad-CAM zoom (MATH §M25) | HoughCircles adaptive crop (MATH §M24)<br>— Grad-CAM removed |
| Stage 1 threshold | 0.70                                                            | 0.70 (unchanged)                                             |
| Stage 2           | 5-fold ensemble, 256×256 crop                                   | Same                                                         |
| TTA               | 8 D4 views                                                      | 8 D4 views (unchanged)                                       |
| Platt calibration | Loaded from .pkl                                                | Loaded from .pkl (unchanged)                                 |
| mark_step         | —                                                               | xm.mark_step() after each inference batch                    |

HoughCircles Crop — TPU (MATH §M24):

```
blurred = GaussianBlur(gray, (9,9), sigma=2)
circles = HoughCircles(blurred, HOUGH_GRADIENT,
dp=1.2, minDist=min(h,w)//2,
param1=100, param2=30,
minRadius=int(min(h,w)*0.15),
maxRadius=int(min(h,w)*0.55))
if circles: cx,cy,r = circles[0,0] else: fallback to centre
```

■ **WARNING** Grad-CAM removed from TPU: dynamic backward hooks break XLA graph tracing. Each Grad-CAM call would trigger full XLA recompilation, making inference extremely slow.

## 23 Pseudo-Labeling

### MATH §M26

After training 5 folds, the ensemble guesses labels for test images. High-confidence guesses are added to training data and a master model is retrained.

```
Confidence filter (MATH §M26):  
p_i = (1/K) * sum_k p_{i,k} K=5 folds  
Keep if: p_i >= 0.95 (pseudo-label 1) OR p_i <= 0.05 (pseudo-label 0)  
Minimum 100 pseudo-labels required  
  
Split order (BUG #9 FIX):  
1. Split ORIGINAL data first → pure ground-truth val set  
2. Append pseudo-labels ONLY to training set  
(Wrong order caused falsely high F1 ~0.99 on val)
```

✓ **FIX BUG #9 FIXED:** Original code mixed pseudo-labels with original data THEN split — putting pseudo-labeled test images in the validation set. The model was evaluated on its own guesses.

## 24 Evaluation & Metrics Report

MATH §M21,M22

`run_evaluation()` auto-detects the best fold checkpoint, runs inference with TTA, sweeps thresholds, and prints a full report.

```
Output includes:  
Accuracy | F1 | Precision | Recall  
TP (Correct Faulty) | TN (Correct Good)  
FP (False Alarms) | FN (Missed Defects)  
+ Worst misclassifications (most confidently wrong)
```

## 25 Known Bugs Fixed — All 24 Issues

| #   | Bug                                               | Impact               | Fix                                                    |
|-----|---------------------------------------------------|----------------------|--------------------------------------------------------|
| #1  | import argparseparser syntax error                | Crash                | Split into import argparse / parser = ArgumentParser() |
| #2  | SwinBridge assumed (B,N,C) not (B,H,W,C)          | Crash                | Changed to x.permute(0,3,1,2) for 4D Swin output       |
| #3  | Missing CBAM, AttentionPool2d, _get_out_channels  | Crash                | Restored from backup notebook                          |
| #4  | BGR→RGB not converted (OpenCV→PIL→model)          | Wrong predictions    | Added cv2.cvtColor(img_bgr, COLOR_BGR2RGB)             |
| #5  | Inference used training transform (random ops)    | Non-deterministic    | Changed to _make_val_transform()                       |
| #6  | Fold detection at module level (IndentationError) | Crash                | Moved inside function                                  |
| #7  | _make_loaders called without img_size             | Missing resize       | Added args.img_size argument                           |
| #8  | Stage 1 doubled imbalance: oversample + pw=9.0    | All preds defective  | Set pos_weight=1.0 when oversampling                   |
| #9  | Pseudo-label split included pseudo in val         | False F1 ~0.99       | Split original first, append pseudo to train only      |
| #10 | 'optimizer' in locals() not in ckpt               | No optimizer restore | Fixed condition                                        |
| #11 | TTA block 20-space indent                         | IndentationError     | Fixed indent level                                     |
| #12 | Training depended on undefined globals            | Crash                | Passed as parameters                                   |
| #13 | Duplicate --seed in argparse                      | Confusing            | Removed duplicate                                      |
| #14 | Hardcoded val_split_fold_1.csv                    | Wrong file           | Dynamic fold detection                                 |
| #15 | Optimizer restored unconditionally on resume      | Phase mismatch crash | Only restore when phase matches                        |
| #16 | Hardcoded crop coords (1280×1024 only)            | Wrong crop           | Relative coordinates                                   |
| #17 | Batch cap at 32                                   | Underutilised GPU    | Removed cap                                            |
| #18 | raise FileNotFoundError when no fold CSVs         | Ugly crash           | Graceful print + return                                |
| #19 | Duplicate STAGE1_IMG_SIZE                         | Confusing            | Commented duplicate                                    |
| #20 | No RAM warning for large cache                    | OOM crash            | Warning if >5000 images                                |
| #21 | ckpt variable scope fragile                       | NameError            | Initialize ckpt={} unconditionally                     |
| #22 | Warmup LR negative on tiny datasets               | Wrong LR             | Guard warmup_steps > 0                                 |
| #23 | Dead SwinSequenceAdapter class                    | Dead code            | Removed                                                |

|         |                               |                  |         |
|---------|-------------------------------|------------------|---------|
| #2<br>4 | Duplicate if __name__ entries | Double execution | Removed |
|---------|-------------------------------|------------------|---------|

## 26 OHEM — Online Hard Example Mining

### MATH §M9

OHEM dynamically keeps only the hardest 70% of examples per batch, forcing the model to focus on micro-scratches it keeps getting wrong.

```
# MATH §M9
losses = [ L_total(z_bin_i, z_sev_i, y_i) for i in 0..B-1 ]
k = max(1, floor(B * 0.70)) # keep top 70%
hard_losses = top_k(losses, k, largest=True)
L_OHEM = mean(hard_losses)

keep_frac effect:
1.0 = normal loss (no OHEM)
0.7 = recommended (drop easiest 30%)
0.5 = aggressive
<0.4 = unstable
```

■ **NOTE** VRAM impact: zero extra. Time: +2-3% per epoch. Expected gain: model concentrates on subtle micro-scratches and borderline cases.



## 27 Grad-CAM Dynamic Cropping (GPU v4.0 Only)

TPU §T10

MATH §M25

Grad-CAM produces a spatial heatmap showing which pixels influenced the defect decision. Used at inference to zoom into the suspected defect area before the ensemble runs.

```
# MATH §M25
alpha_k = (1/H'*W') * sum_{i,j} dL/dA^k_{i,j} 'GAP of gradients'
CAM = ReLU( sum_k alpha_k * A^k ) shape: (H', W')
CAM_norm = (CAM - min) / (max - min + 1e-8)
Resize CAM_norm to full image (bilinear)

Hotspot: (cy,cx) = argmax(CAM_norm)
Crop: half = min(H,W) * (0.5+0.15)
patch = img[cy-half:cy+half, cx-half:cx+half]
```

Confidence gate: Grad-CAM only applied when B0 prob  $\geq 0.4$ . Otherwise falls back to standard rim crop.

■ **WARNING** Removed from TPU: XLA traces computation graphs statically. Backward hooks + `torch.enable_grad()` inside `no_grad` context causes graph inconsistencies and triggers full recompilation per image.

## 28 Glossary

- Accuracy** — Fraction of all predictions that are correct. Misleading on imbalanced data.
- AdamW** — Optimizer with decoupled weight decay. Used for all trainable parameters.
- AMP** — Automatic Mixed Precision. Uses float16 or bfloat16 to halve memory and double speed.
- ASL** — Asymmetric Loss.  $\gamma_{neg}=4$  aggressively suppresses easy negatives.
- Attention** — Mechanism letting the model focus on important channels and pixels.
- AUC-ROC** — Threshold-independent ranking quality metric. 1.0=perfect, 0.5=random.
- Augmentation** — Creating modified copies of training images to improve generalisation.
- Backbone** — Pretrained feature extractor (Swin, EfficientNet, ConvNeXt).
- BatchNorm** — Normalises layer outputs to mean 0, std 1. Running stats used at inference.
- bfloat16** — 16-bit float with 8 exponent bits. Same dynamic range as float32. No GradScaler needed.
- CBAM** — Convolutional Block Attention Module. Channel + spatial attention gates.
- Cascade** — Two-stage pipeline: B0 fast filter → EnsembleV6 for uncertain cases.
- Cosine Annealing** — LR schedule: smooth decay from  $lr\_max$  to  $lr\_min$  over  $T\_max$  epochs.
- Cross-Attention** — Multi-head attention letting each backbone token attend to the other two.
- CutMix** — Paste a rectangular crop from image B onto image A; mix labels by area ratio.
- DistributedSampler** — Divides dataset equally across TPU chips so no two chips see the same sample.
- Dropout** — Randomly zero neurons during training. Prevents memorisation.
- EfficientNet** — CNN family by Google (Neural Architecture Search). B0=5M params, B3=12M.
- EMA** — Exponential Moving Average. Shadow model = slow-running average of weights.
- Ensemble** — Average predictions from multiple models for higher accuracy.
- Epoch** — One complete pass through the training dataset.
- F1 Score** — Harmonic mean of Precision and Recall. Primary metric. Range 0–1.
- Focal Loss** — Down-weights easy examples; focuses gradient on hard  $(1-pt)^\gamma$  factor.
- Fold** — One split in k-fold cross-validation.
- GELU** — Gaussian Error Linear Unit. Smooth activation used in Transformer layers.
- GPU** — Graphics Processing Unit. Runs CUDA. Used in v4.0 notebook.
- Grad-CAM** — Gradient-weighted Class Activation Map. Heatmap of important pixels. GPU only.
- Gradient Clip** — Scale all gradients by  $\max\_norm/global\_norm$  when norm exceeds  $\max\_norm=0.5$ .
- GradScaler** — Multiplies loss by  $2^{16}$  before backward to prevent float16 underflow.
- HoughCircles** — Classical CV algorithm detecting circles in edge images. Used in TPU crop.
- Label Smoothing** — Soften labels:  $0 \rightarrow \epsilon/2$ ,  $1 \rightarrow 1-\epsilon/2$ . Prevents overconfidence.
- Learning Rate** — Step size for weight updates. Too large=unstable, too small=slow.
- Logit** — Raw model output before sigmoid. Range:  $(-\infty, +\infty)$ .
- MixUp** — Blend two images and labels:  $x = \lambda x_A + (1-\lambda)x_B$ .
- MpDeviceLoader** — TPU: wraps DataLoader to prefetch batches to XLA device asynchronously.
- OHEM** — Online Hard Example Mining. Keep top-k highest-loss examples per batch.
- Oversample** — Duplicate minority class examples to achieve 50/50 class balance.
- Platt Scaling** — Fit logistic regression on val outputs to calibrate probabilities.
- Precision** —  $TP/(TP+FP)$ . Of predicted defectives, how many are real?
- Pseudo-Label** — High-confidence model predictions used as training labels for unlabeled data.
- Recall** —  $TP/(TP+FN)$ . Of all real defectives, how many were caught?
- SWA** — Stochastic Weight Averaging. Arithmetic mean of weights over final epochs.
- Sigmoid** —  $\sigma(x) = 1/(1+\exp(-x))$ . Converts logit to probability in  $(0,1)$ .

**Swin Transformer** — Vision Transformer with Shifted Windows for efficient global attention.

**TTA** — Test-Time Augmentation. Average predictions over 8 D4 symmetry transforms.

**TPU** — Tensor Processing Unit (Google). 8 chips via torch\_xla / XLA on Kaggle.

**Threshold** — Cutoff probability for classification. Swept 0.05–0.95 to maximise F1.

**xm.mark\_step()** — TPU: flush the pending XLA computation graph and execute it.

**xm.optimizer\_step()** — TPU: all-reduce gradients across chips + mark\_step + update weights.

**xmp.spawn()** — TPU: launch one worker process per chip (8 total on v5e-8).

## PART 2

# TPU Changes

Every change from GPU v4.0 to the TPU notebook — matched to notebook sections

## T1 Overview — 10 Core Changes at a Glance

| Feature                    | GPU Notebook v4.0                                       | TPU Notebook                                               |
|----------------------------|---------------------------------------------------------|------------------------------------------------------------|
| Compute backend            | <code>torch.cuda / CUDA</code>                          | <code>torch_xla / XLA (Google)</code>                      |
| Device creation            | <code>torch.device('cuda')</code>                       | <code>xm.xla_device()</code>                               |
| Batch size                 | 2                                                       | 192 (8 chips × 24)                                         |
| Gradient accum             | 32 steps                                                | 1 step (real batch 192)                                    |
| Precision                  | <code>bfloat16</code> via <code>autocast('cuda')</code> | <code>bfloat16</code> native; <code>autocast('xla')</code> |
| Loss scaling               | GradScaler for float16                                  | Removed — <code>bfloat16</code> needs no scaler            |
| Optimizer step             | <code>optimizer.step()</code>                           | <code>xm.optimizer_step(optimizer)</code>                  |
| Data prefetch              | <code>DataLoader pin_memory=True</code>                 | <code>pl.MpDeviceLoader</code> wraps each loader           |
| Multi-chip                 | Single GPU, single process                              | <code>xmp.spawn</code> — one process per chip              |
| Grad-CAM crop              | Dynamic backward-hook heatmap                           | Removed; <code>HoughCircles</code> used instead            |
| Channels-last              | <code>True</code>                                       | <code>False</code> (not applicable on TPU)                 |
| <code>torch.compile</code> | Optional flag                                           | Disabled — unsupported on XLA                              |
| Grad checkpoint            | <code>True</code> (saves VRAM)                          | <code>False</code> (16 GB/chip enough at batch 192)        |

## T2 Environment Setup — torch\_xla Replaces CUDA

### GPU §3

The GPU notebook imports PyTorch + CUDA config. The TPU notebook removes CUDA config and imports the full XLA API.

```
# New TPU imports
import torch_xla
import torch_xla.core.xla_model as xm
import torch_xla.runtime as xr
import torch_xla.distributed.xla_multiprocessing as xmp
import torch_xla.distributed.parallel_loader as pl

def get_device(): return xm.xla_device()
def get_num_tpu_chips(): return xr.world_size() # 8
```

T3 Configuration — Batch, Accumulation, Precision

GPU §4

The largest config changes: batch explodes from 2 → 192; accumulation drops from 32 → 1; bf16 becomes always-on.

| Feature         | GPU Notebook v4.0 | TPU Notebook                 |
|-----------------|-------------------|------------------------------|
| --batch         | 2                 | 192                          |
| --accum-steps   | 32                | 1                            |
| --bf16          | False             | True (always on, TPU native) |
| --label-smooth  | 0.02              | 0.01                         |
| --mixup-alpha   | 0.10              | 0.20                         |
| --focal-gamma   | 2.0               | 2.5                          |
| --ema-decay     | 0.9998            | 0.9999                       |
| --channels-last | True              | False                        |
| --compile       | False             | False (XLA incompatible)     |
| --workers       | 2                 | 4                            |
| STAGE1_BATCH    | 8                 | 128                          |

T4

Stage 1 — Bigger Batches, Native bfloat16

GPU \$5

| Feature          | GPU Notebook v4.0                        | TPU Notebook                                             |
|------------------|------------------------------------------|----------------------------------------------------------|
| Batch size       | 8                                        | 128                                                      |
| autocast         | <code>torch.autocast('cuda', ...)</code> | <code>torch.autocast('xla', dtype=torch.bfloat16)</code> |
| Optimizer step   | <code>optimizer.step()</code>            | <code>xm.optimizer_step(optimizer)</code>                |
| Save checkpoint  | <code>torch.save(state, path)</code>     | <code>xm.save(state, path)</code>                        |
| pin_memory       | True                                     | False                                                    |
| After EMA update | —                                        | <code>xm.mark_step()</code>                              |



T5 Data Loading — DistributedSampler & MpDeviceLoader

GPU §9

8 chips must each see a disjoint 1/8 of data. DistributedSampler handles partitioning; MpDeviceLoader handles asynchronous prefetch.

| Feature            | GPU Notebook v4.0          | TPU Notebook                      |
|--------------------|----------------------------|-----------------------------------|
| pin_memory         | True                       | False                             |
| Shuffle            | shuffle=True in DataLoader | DistributedSampler(shuffle=True)  |
| Sampler            | WeightedRandomSampler      | DistributedSampler                |
| Device loader      | DataLoader directly        | pl.MpDeviceLoader(loader, device) |
| persistent_workers | True                       | False                             |
| prefetch_factor    | 2                          | Removed                           |

T6

Augmentations — Two New Transforms

GPU §6,7

All 5 original custom augmentations preserved. Two new standard augmentations added to the heavy Phase 3 pipeline:

| New Transform            | Mechanism                | Simulates                                  |
|--------------------------|--------------------------|--------------------------------------------|
| A.GridDistortion(p=0.25) | Local grid-based warping | Lens distortion, geometric variations      |
| A.RandomShadow(p=0.20)   | Random dark polygon      | Factory lighting shadows, fixture brackets |

■ **NOTE** All augmentations remain CPU-only. TPU MXUs handle matrix ops; image transforms stay on the CPU data workers.

T7 Rim Cropping — HoughCircles Replaces Fixed Coords

GPU §22      MATH §M24

| Feature                   | GPU Notebook v4.0                                       | TPU Notebook                       |
|---------------------------|---------------------------------------------------------|------------------------------------|
| Method                    | Fixed relative coordinates derived from camera position | HoughCircles adaptive detection    |
| Adapts to off-centre rims | No — always crops at same relative position             | Yes — finds actual circle in image |
| Grad-CAM zoom             | Present (prob >= 0.4 → zoom to hotspot)                 | Removed (XLA incompatible)         |
| Fallback                  | —                                                       | Centre crop if HoughCircles fails  |
| Margin                    | 0.05 (5%)                                               | 0.10 (10%)                         |

T8 Training Loop — xm.\* API Replaces torch.cuda.\*

GPU §21

| Feature         | GPU Notebook v4.0                                                                           | TPU Notebook                                              |
|-----------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| autocast        | <code>torch.autocast('cuda', dtype=...)</code>                                              | <code>torch.autocast('xla', dtype=bfloat16)</code>        |
| GradScaler      | <code>scaler.scale(loss).backward()</code><br><code>scaler.step(opt) scaler.update()</code> | Removed entirely                                          |
| Optimizer step  | <code>optimizer.step()</code>                                                               | <code>xm.optimizer_step(optimizer)</code>                 |
| After EMA       | —                                                                                           | <code>xm.mark_step()</code>                               |
| Memory report   | <code>torch.cuda.memory_allocated()</code>                                                  | <code>xm.get_memory_info(device)</code>                   |
| Checkpoint save | <code>torch.save()</code>                                                                   | <code>xm.save()</code>                                    |
| Master rank     | Single process (always master)                                                              | <code>xm.is_master_ordinal()</code> guards prints/saves   |
| Early stop sync | Single process (no sync needed)                                                             | <code>xm.mesh_reduce('key', val, any)</code> across chips |

```
# Distributed early stopping – all chips agree
do_break = xm.mesh_reduce('early_stop', do_break, lambda xs: any(xs))
```

T9

Validation & BN Recalibration

GPU §16,17

Structurally identical to GPU. Two TPU-specific changes:

| Change          | Detail                                                                  |
|-----------------|-------------------------------------------------------------------------|
| Loader wrapping | All loaders wrapped with pl.MpDeviceLoader before iteration             |
| Graph flush     | xm.mark_step() called after every validation and recalibration batch    |
| Threshold sweep | Still runs on CPU tensors — small enough that XLA overhead not worth it |

T1  
0

Inference Pipeline — Grad-CAM Removed

GPU \$22,27

Cascade structure preserved. Stage 1 threshold, TTA, Platt calibration all unchanged. Grad-CAM removed; HoughCircles replaces it.

| Feature   | GPU Notebook v4.0                      | TPU Notebook               |
|-----------|----------------------------------------|----------------------------|
| Rim crop  | Fixed coords + Grad-CAM (if prob>=0.4) | HoughCircles adaptive      |
| Grad-CAM  | torch.enable_grad() inside no_grad     | Removed — XLA incompatible |
| mark_step | —                                      | After each inference batch |
| save      | torch.save()                           | xm.save()                  |

■ **WARNING** Why Grad-CAM fails on XLA: (1) dynamic backward hooks break static graph tracing. (2) Mixing no\_grad/enable\_grad contexts causes graph inconsistencies. (3) Each call triggers full XLA recompilation.

T1  
1

## Multi-Process Entry Point — xmp.spawn

The biggest structural addition. GPU notebook: single process. TPU notebook: 8 worker processes, one per chip.

```
# Entry point
xmp.spawn(_train_worker,
args=(args, all_samples, assets),
nprocs=get_num_tpu_chips(), # 8
start_method='spawn')

# Inside _train_worker(rank, args, all_samples, assets):
device = xm.xla_device() # this chip
# ... k-fold loop ...
xm.rendezvous('fold_N_done') # barrier: wait for all 8 chips

# Gradient sync happens inside xm.optimizer_step():
# all-reduce gradients across chips → update weights
# Effective batch = 8 × 192 = 1,536 images per step
```

■ **NOTE** Only chip 0 (`xm.is_master_ordinal()`) prints logs and saves checkpoints. All chips participate in training and gradient synchronisation.

T1  
2

## Complete Hyperparameter Reference

| Feature            | GPU Notebook v4.0     | TPU Notebook               |
|--------------------|-----------------------|----------------------------|
| --batch            | 2                     | 192                        |
| --lr               | 5e-5                  | 5e-5                       |
| --img-size         | 384                   | 384                        |
| --accum-steps      | 32                    | 1                          |
| --label-smooth     | 0.02                  | 0.01                       |
| --mixup-alpha      | 0.10                  | 0.20                       |
| --focal-gamma      | 2.0                   | 2.5                        |
| --ema-decay        | 0.9998                | 0.9999                     |
| --grad-clip        | 0.5                   | 0.5                        |
| --dropout-rate     | 0.35                  | 0.35                       |
| --val-split        | 0.12                  | 0.12                       |
| --phase1-epochs    | 5                     | 5                          |
| --phase2-epochs    | 8                     | 8                          |
| --phase3-epochs    | 27                    | 27                         |
| --swa-start        | 30                    | 30                         |
| --early-stop       | 12                    | 12                         |
| --folds            | 5                     | 5                          |
| --bf16             | False                 | True                       |
| --channels-last    | True                  | False                      |
| --compile          | False                 | False                      |
| --grad-checkpoint  | True                  | False                      |
| --workers          | 2                     | 4                          |
| STAGE1_BATCH       | 8                     | 128                        |
| STAGE1_EPOCHS      | 15                    | 15                         |
| STAGE1_THRESHOLD   | 0.70                  | 0.70                       |
| GridDistortion     | Absent                | A.GridDistortion(p=0.25)   |
| RandomShadow       | Absent                | A.RandomShadow(p=0.20)     |
| Rim crop           | Fixed relative coords | HoughCircles adaptive      |
| Grad-CAM           | Present               | Removed                    |
| DistributedSampler | No                    | Yes                        |
| MpDeviceLoader     | No                    | Yes                        |
| xm.mark_step()     | No                    | After every batch/EMA/save |



|                     |    |                           |
|---------------------|----|---------------------------|
| xm.optimizer_step() | No | Replaces optimizer.step() |
| xm.save()           | No | Replaces torch.save()     |
| xmp.spawn()         | No | Entry point (8 processes) |

## PART 3

# Mathematics Reference

Every formula from both notebooks — matched to the sections that use them

## M1 Image Normalisation

**GPU §3**

Applied to every image before entering the network. Matches the statistics the pretrained backbone was trained on (ImageNet-1k).

```
x_norm[c] = (x[c] / 255 - mean[c]) / std[c]
```

```
mean = [0.485, 0.456, 0.406] (R, G, B)
```

```
std = [0.229, 0.224, 0.225]
```

After normalisation: each channel has approx mean=0, var=1.

## M2 Data Augmentation Mathematics

GPU \$6,7

### M2a — Specular Highlight (2D Gaussian Ellipse)

```
mask(x,y) = exp( -[ ((x-cx)/rx)^2 + ((y-cy)/ry)^2 ] )
pixel_out[c] = clip( pixel_in[c] + mask*intensity*255, 0,255 )
cx~U(W/4,3W/4) cy~U(H/4,3H/4) intensity~U(0.2,0.7)
```

### M2b — SectorMask (Pie Slice)

```
r_bound = ceil( sqrt((W/2)^2+(H/2)^2) ) + 1
start_angle~U(0,360) sweep_angle~U(10,60)
```

### M2c — Vignette (Normalised Radial Distance)

```
dist(x,y) = sqrt(((x-W/2)/(W/2))^2 + ((y-H/2)/(H/2))^2)
vignette = 1 - clip(dist*strength, 0, 0.6) strength~U(0.3,0.7)
pixel_out = pixel_in * vignette
```

### M2d — Ring Light (Annular Mask)

```
dist(x,y) = sqrt((x-W/2)^2+(y-H/2)^2)
ring = 1 if r1<=dist<=r2 else 0
r1~U(0.2,0.4)*min(W,H) r2=r1+U(5,20)
pixel_out = clip(pixel_in + ring*tint*30, 0,255)
```

### M2e — ScratchSimulation (Trigonometric Line)

```
x2 = int(x1 + length*cos(angle))
y2 = int(y1 + length*sin(angle))
length~U(20,W/2) angle~U(0,pi)
```

## M3 Oversampling & Class Weighting

GPU §2,9

```
# M3a Oversampling
n_extra = floor(n0*r) - n1 r=1.0
extra[i] ~ UniformChoice(minority) # with replacement

# M3b WeightedRandomSampler
w0=1/n0 w1=1/n1
P(sample i) = w[label_i] / sum_j w[label_j]

# M3c Positive weight (when NOT oversampling)
pos_weight = n0/n1
BCE_weighted = -[pw*y*log(sigma(z)) + (1-y)*log(1-sigma(z))]
When oversampling: pos_weight = 1.0
```

## M4 Attention Mechanisms — CBAM

GPU §10

```
# Channel Attention
F_avg=AvgPool(X) F_max=MaxPool(X) →(B,C)
MLP(v)=W2*ReLU(W1*v) W1:(C,C/16) W2:(C/16,C)
M_c = sigma(MLP(F_avg)+MLP(F_max)) →(B,C)
X' = X * M_c.unsqueeze(-1).unsqueeze(-1)

# Spatial Attention
sp_in = cat(mean(X',C), max(X',C)) →(B,2,H,W)
M_s = sigma(Conv7x7(sp_in)) →(B,1,H,W)
X'' = X' * M_s
```

## M5 Attention Pooling (AttentionPool2d)

GPU §10

```
# Standard GAP (not used for defect features)
GAP(X) = (1/H*W) * sum_{i,j} X[:, :, i, j]

# Soft-Attention Pooling (used in ensemble)
scores = Conv1x1(X) → (B, 1, H, W)
w = softmax(scores.flatten(dim=2)) → (B, 1, H*W)
AttPool(X) = sum_k w[:, 0, k] * X_flat[:, :, k] → (B, C)
```

## M6 Cross-Backbone Multi-Head Attention

GPU §11

```
# Linear projection to common dim
f1'=w1*f1 f2'=w2*f2 f3'=w3*f3 →(B,256)
seq = stack([f1',f2',f3'], dim=1) →(B,3,256)

# Scaled Dot-Product Attention (per head k)
d_head=256/8=32
head_k = softmax(Q_k*K_k^T / sqrt(32)) * V_k
MultiHead = concat(head_1..head_8) * w_0 →(B,3,256)

# Fusion
fused = LayerNorm(mean(MultiHead, dim=1)) →(B,256)
LayerNorm(x) = (x-mean(x))/sqrt(var(x)+1e-5) * gamma + beta
```



## M7 Activation Functions

**GPU §10,11,12**

```
# Sigmoid – binary probability
sigma(x) = 1/(1+exp(-x)) range:(0,1)

# GELU – Transformer-style activation
GELU(x) = x*Phi(x) Phi=standard_normal_CDF
approx: 0.5*x*(1+tanh(sqrt(2/pi)*(x+0.044715*x^3)))

# ReLU – CNN activation
ReLU(x) = max(0,x)

# Softmax – attention weights
softmax(x)_i = exp(x_i) / sum_j exp(x_j)
```

## M8 Loss Functions — Full Derivations

### GPU §12

```
# M8a Base BCE
BCE(logit,y) = -[y*log(sigma(z)) + (1-y)*log(1-sigma(z))]
```

# M8b Label Smoothing

$$y_{\text{smooth}} = y(1-\epsilon) + 0.5\epsilon \quad \epsilon=0.02(\text{GPU})/0.01(\text{TPU})$$

# M8c Focal Loss

$$p_t = p \cdot y + (1-p) \cdot (1-y) \quad p=\text{sigma}(\text{logit})$$
$$\text{Focal} = (1-p_t)^\gamma \cdot \text{BCE} \quad \gamma=2.0(\text{GPU})/2.5(\text{TPU})$$

# M8d Asymmetric Loss

$$p_{\text{clip}} = \text{clip}(p+0.05, \text{max}=1-\epsilon)$$
$$L_{\text{pos}} = -y \cdot \log(\text{clip}(p, \text{min}=\epsilon))$$
$$L_{\text{neg}} = -(1-y) \cdot \log(1-p_{\text{clip}})$$
$$w_{\text{asl}} = (1-p_t)^\gamma_{\text{pos}}(y \geq 0.5) \text{ or } (1-p_t)^\gamma_{\text{neg}}(y < 0.5)$$
$$\gamma_{\text{pos}}=1 \quad \gamma_{\text{neg}}=4$$
$$\text{ASL} = -(L_{\text{pos}}+L_{\text{neg}}) \cdot w_{\text{asl}}$$

# M8e Combined Loss

$$L_{\text{bin}} = 0.60 \cdot \text{Focal} + 0.40 \cdot \text{ASL}$$
$$L_{\text{sev}} = (\text{sigma}(z_{\text{sev}})-y)^2$$
$$L_{\text{total}} = L_{\text{bin}} + 0.25 \cdot L_{\text{sev}}$$

## M9 Online Hard Example Mining (OHEM)

GPU §26

```
losses = [L_total(z_bin_i, z_sev_i, y_i) for i in 0..B-1]
k = max(1, floor(B * 0.70))
hard_losses = top_k(losses, k, largest=True)
L_OHEM = mean(hard_losses)
```

M1  
0

## Exponential Moving Average (EMA)

GPU §13

```
d_t = min(decay, (1+t)/(10+t)) # warmup ramp  
shadow[t+1] = d_t*shadow[t] + (1-d_t)*theta[t]
```

```
decay = 0.9998(GPU) / 0.9999(TPU)  
Effective window = 1/(1-decay)  
GPU: 5,000 steps TPU: 10,000 steps
```

M1  
1

## MixUp &amp; CutMix

## GPU §14

```
lambda ~ Beta(alpha,alpha) alpha=0.10(GPU)/0.20(TPU)

MixUp:
x_mix = lambda*x + (1-lambda)*x[idx]
y_mix = lambda*y + (1-lambda)*y[idx]

CutMix:
cut_r=sqrt(1-lambda) cut_h=H*cut_r cut_w=W*cut_r
x_mix[:, :, y1:y2, x1:x2] = x[idx, :, :, y1:y2, x1:x2]
lambda_actual = 1-(x2-x1)*(y2-y1)/(H*W)
y_mix = lambda_actual*y + (1-lambda_actual)*y[idx]
```

M1  
2

## Test-Time Augmentation — D4 Dihedral Group

GPU §15

```
D4 = {identity, rot90, rot180, rot270,  
flipH, flipH+rot90, flipV, flipV+rot90}  
  
p_TTA = (1/8) * sum_{g in D4} sigma(model(g(x)))  
y_hat = 1 if p_TTA >= threshold, else 0
```

M1  
3

## Optimiser — AdamW

GPU §19,21

```
m_t = beta1*m_{t-1} + (1-beta1)*g_t beta1=0.9
v_t = beta2*v_{t-1} + (1-beta2)*g_t^2 beta2=0.999
m_hat = m_t/(1-beta1^t)
v_hat = v_t/(1-beta2^t)
theta_{t+1} = theta_t - lr*m_hat/sqrt(v_hat+1e-8) - lr*1e-4*theta_t
(last term = decoupled weight decay)
```

M1  
4

## Learning Rate Schedule — Cosine Annealing

GPU §19

```
lr_t = lr_min + 0.5*(lr_max-lr_min)*(1+cos(pi*t/T_max))  
lr_max=5e-5 lr_min=5e-7 T_max=epochs in phase  
  
t=0: lr = lr_max (full rate)  
t=T_max: lr = lr_min (near zero)
```



M1  
5

## Gradient Clipping

GPU §21

```
g_norm = sqrt( sum_i ||grad_i||^2 ) [global L2 norm]
if g_norm > 0.5:
  grad_i <- grad_i * (0.5/g_norm) for all i
```

M1  
6

## Gradient Accumulation &amp; Effective Batch Size

GPU §21

```
loss_step = L_total(batch) / accum_steps
for step in range(accum_steps):
    loss_step.backward() # accumulates into .grad
    optimizer.step() # one weight update
```

```
effective_batch = batch * accum_steps
GPU: 2*32=64 TPU: 192*1=192
```

Dividing loss by accum\_steps keeps gradient magnitude equivalent to a single pass on the full effective batch.

M1  
7

## Mixed Precision &amp; Loss Scaling

GPU §20

TPU §T3

```
# float16 (GPU Volta/Turing only)
scaled_loss = loss * scale_factor # typically 2^16
scaled_loss.backward() # scaled grads, no underflow
scaler.unscale_(optimizer) # divide grads by scale_factor
clip_grad_norm_(model.params, 0.5)
scaler.step(optimizer)
scaler.update() # adapts scale_factor each step

# bfloat16 (Ampere GPU + TPU)
# 8 exponent bits: same range as float32
# No underflow risk → no GradScaler needed
```

M1  
8

## Batch Normalisation &amp; Recalibration

## GPU §17

```
# Training
mu_B = (1/B)*sum_i x_i
sigma_B = sqrt((1/B)*sum_i(x_i-mu_B)^2 + eps)
x_hat_i = (x_i-mu_B)/sigma_B
y_i = gamma*x_hat_i + beta

# Running stats
mu_run <- (1-m)*mu_run + m*mu_B
sigma_run <- (1-m)*sigma_run + m*sigma_B

# Recalibration after SWA/EMA
Reset mu_run=0, sigma_run=1, n_tracked=0
Set momentum=None → cumulative moving average
Run 25 forward passes → stats reflect EMA model
```

M1  
9

## Stochastic Weight Averaging (SWA)

GPU §21

```
theta_SWA <- theta_SWA + (theta_epoch-theta_SWA)/n_averaged  
= (1/n) * sum_{k=1}^{n} theta_{swa_start+k}
```

```
swa_start = 30 (both notebooks)
```

SWA vs EMA:

SWA = arithmetic mean (all epochs equally weighted)

EMA = exponential mean (recent epochs weighted more)

M2  
0

## Platt Scaling

GPU §18

```
calibrated_prob = sigma(A*raw_prob + B)

Fit by minimising:
sum_i BCE(sigma(A*p_i+B), y_i) over validation set

cal.fit( probs.reshape(-1,1), labels )
A = cal.coef_[0,0] B = cal.intercept_[0]
```

M2  
1

## Evaluation Metrics

GPU §16,24

```
Accuracy = (TP+TN)/(TP+TN+FP+FN)
Precision = TP/(TP+FP)
Recall = TP/(TP+FN)
F1 = 2*P*R/(P+R) = 2*TP/(2*TP+FP+FN)
AUC-ROC = P(score(defective) > score(clean))
```

M2  
2

## Threshold Sweep — Vectorised F1

GPU §16

```
probs (N,) N = validation samples
thrs (91,) linspace(0.05, 0.95, step=0.01)
preds = (probs.unsqueeze(0) >= thrs.unsqueeze(1)) (91,N)
TP = (preds*labels).sum(1)
FP = (preds*(1-labels)).sum(1)
FN = ((1-preds)*labels).sum(1)
P = TP/clip(TP+FP, min=1)
R = TP/clip(TP+FN, min=1)
F1 = 2*P*R/clip(P+R, min=1e-8)
best_threshold = thrs[argmax(F1)]
```



M2  
3

## Cascade Inference Decision Rule

GPU §1,22

```
p_B0 = sigma(B0(full_image))
p_ensemble = mean_k(sigma(Ensemble_k(crop))) # k folds

p_final = p_B0 if p_B0 >= 0.70 (Stage 1 confident)
          = p_ensemble otherwise

y_hat = 1 if p_final >= tau_blend, else 0
tau_blend = mean of per-fold optimal thresholds
```

M2  
4

## Rim Cropping — HoughCircles (TPU)

TPU ST7

```
# Hough Transform accumulator vote
For each edge pixel (x,y) at angle theta:
a = x - r*cos(theta)
b = y - r*sin(theta)
Accumulator[a,b,r] += 1
Circle: (a*,b*,r*) = argmax(Accumulator)

side = 2*r*(1+margin) margin=0.10
crop = img[cy-side//2:cy+side//2, cx-side//2:cx+side//2]
crop resized to (out_size,out_size) via bilinear

Pre-processing: GaussianBlur(kernel=(9,9), sigma=2)
param1=100 (Canny upper) param2=30 (accumulator thr)
```

M2  
5

## Grad-CAM (GPU v4.0 Only)

GPU §27

TPU §T10

```

alpha_k = (1/H'*W') * sum_{i,j} dL/dA^k_{i,j} 'GAP of gradients'
CAM = ReLU(sum_k alpha_k * A^k) shape:(H',W')
CAM_norm= (CAM-min)/(max-min+1e-8)
Resize CAM_norm to full image (bilinear)

Hotspot: (cy,cx) = argmax(CAM_norm)
half = min(H,W)*(0.5+0.15)
patch = img[cy-half:cy+half, cx-half:cx+half]
Confidence gate: only if p_B0 >= 0.4

```

M2  
6

## Pseudo-Labelling Confidence Filter

GPU §23

```
p_i = (1/K)*sum_{k=1}^K p_{i,k} K=5 folds
```

Keep image  $i$  as pseudo-label if:

```
p_i >= 0.95 → y=1 (defective)
```

```
p_i <= 0.05 → y=0 (clean)
```

Minimum 100 pseudo-labels required to proceed.

Ambiguous ( $0.05 < p < 0.95$ ) → discarded.

M2  
7

## Vignette, Ring &amp; Specular Highlight — Full Geometry

GPU §6

**Vignette factor (MATH §M2c, full):**

```

d(x,y) = sqrt(((x-W/2)/(W/2))^2 + ((y-H/2)/(H/2))^2)
corners: d=sqrt(2)≈1.414 edges: d=1
v(x,y) = 1 - clip(d*s, 0, 0.6) s~U(0.3,0.7)
pixel_out = pixel_in * v
Max darkening at corners: 1-0.6=0.4 (40% original brightness)

```

**Annular ring mask (full):**

```

d(x,y) = sqrt((x-W/2)^2+(y-H/2)^2) [pixels, unnormalised]
ring(x,y)= 1 if r1<=d<=r2 else 0
r1~U(0.2,0.4)*min(W,H) r2=r1+U(5,20)
pixel_out= clip(pixel_in+ring*tint*30, 0,255)
tint[c]~U(0.8,1.2) per channel

```

**Specular highlight (full):**

```

mask(x,y) = exp(-[((x-cx)/rx)^2+((y-cy)/ry)^2])
cx~U(W/4,3W/4) cy~U(H/4,3H/4)
rx~U(W/10,W/4) ry~U(H/10,H/4)
intensity~U(0.2,0.7)
pixel_out[c]=clip(pixel_in[c]+mask*intensity*255, 0,255)

```

## End of Document

Rim Defect Detection — Complete Unified Reference GPU Notebook v4.0 · TPU Notebook · Mathematics Reference All three documents merged, cross-referenced, and matched Updated May 2026